# Classification II

Last week we covered the univariate Gaussian classifier for image segmentation. This week, we will extend this classifier to utilize multiple features using a *multivariate Gaussian classifier*. The multivariate case is a straight forward generalization of the univariate case to higher dimensions, so this note will be quite similar the Classification I notes.

## Multivariate Gaussian classifier

As before, the task is to assign a class $c \in \mathcal{C}$ to every pixel $s$ in an image $\mathcal{S}$. Each pixel $s$ will now be associated with $n_x$ observable (or computable) features, which will be collected in a vector $x \in \mathbb{R}^{n_x}$. We will use these features simultainously to determine in which class $s$ belongs, or equivivalently, in which class the feature vector $x$ will belong. We will let $\mathcal{X}$ denote the set of feature vectors associated with the image $\mathcal{S}$ such that each $s \in \mathcal{S}$ will have a unique feature vector $x \in \mathcal{X}$. Note that even though each pixel has an assosiated feature vector, the value of the feature vector can be shared by multiple pixels.

### Probability model

For each $s$ we will model its associated feature vector as a continuous random vector $X$ which range is in $\mathbb{R}^{n_x}$. Likewise, we will model the class as a discrete random variable $C$ which range is in $\mathcal{C}$. $C$ will have an *a priori* probability density $p_C(c) = Pr(C = c)$ which is the probability of labeling $s$ as belonging to $c$ before any features are observed. Likewise, we have the probability density of $X$, $f_X(x) = Pr(X = x)$, which, by total probability, can be written as

$$\begin{aligned} f_X(x) &= \sum_{c \in \mathcal{C}} f_{X,C}(x, c) \\ &= \sum_{c \in \mathcal{C}} f_{X|C=c}(x) p_C(c), \end{aligned}$$

where $f_{X,C}(x, c)$ is the joint probability of $X$ taking the value $x$ and $C$ taking the value $c$.

We now define the likelihood $f_{X|C=c}(x) = Pr(X = x|C = c)$ which is the probability of $X$ taking the value of $x$ if it indeed was in class $c$. With this, we can now define the *a posteriori* probability density

$$\begin{aligned} f_{C|X=x}(c) &= \frac{f_{X|C=c}(x) p_C(c)}{f_X(x)} \\ &= \frac{f_{X|C=c}(x) p_C(c)}{\sum_{c' \in \mathcal{C}} f_{X|C=c'}(x) p_C(c')}. \end{aligned}$$

Note that the denominator is not dependent on the particular class $c$, and we can view this as simply a normalization factor.

$x$ $a : \mathbb{R}^{n_x} \to \mathcal{C}$

mapping from an observed feature vector $x$ to a class $c$,

$$\begin{aligned} a(x) &= \arg\max_{c \in \mathcal{C}} \{f_{C|X=x}(c)\} \\ &= \arg\max_{c \in \mathcal{C}} \{f_{X|C=c}(x) p_C(c)\} \\ &= \arg\max_{c \in \mathcal{C}} \{\log(f_{X|C=c}(x)) + \log(p_C(c))\}. \end{aligned}$$

## Gaussian distribution

The previous section was very similar to the corresponding discussion in the notes about the univariate classifier. This section is also a straight forward generalization of the univariate case.

We assume that $C$ is uniformly distributed over $\mathcal{C}$,

$$p_C(c) = \begin{cases} 1/n_c, & c \in \mathcal{C} \\ 0, & c \notin \mathcal{C}, \end{cases}$$

where $n_c$ are the number of classes in $\mathcal{C}$.

The conditional likelihood will be a multivariate gaussian $(X|C = c) \sim \mathcal{N}_{n_x}(x; \mu_c, \Sigma_c)$

$$f_{X|C=c}(x) = (2\pi)^{-n_x/2} (\det(\Sigma_c))^{-1/2} \exp\left\{ -\frac{1}{2}(x - \mu_c)^\intercal \Sigma_c^{-1} (x - \mu_c) \right\}$$

where $\mu_c$ is the feature mean vector of class $c$ and $\Sigma_c$ is the feature *covariance matrix* of class $c$, and these are the variables that are to be estimated in the training process. $v^\intercal$ denotes the *transpose* of the vector $v$, and the normal vectors are column vectors such that the gaussian kernel is just a scalar.

Note that the discrimination function $a$ is not dependent on the marginal density of $X$, so, as mentioned above, we will not specify it further. Since we have now described our distributions, we can revisit the discrimination function

$$\begin{aligned} a(x) &= \arg\max_{c \in \mathcal{C}} \{\log(f_{X|C=c}(x)) + \log(p_C(c))\} \\ &= \arg\max_{c \in \mathcal{C}} \{-\frac{1}{2}\log(\det(\Sigma_c)) - \frac{1}{2}(x - \mu_c)^\intercal \Sigma_c^{-1} (x - \mu_c)\} \end{aligned}$$

where the last representation is what is actually implemented.

## Training

To be able to determine $\mu_c$ and $\Sigma_c$, we need to have a training set $\mathcal{X}_c^t \subset \mathcal{X}$ for each class label $c$

$$\mathcal{X}_c^t = \{x \in \mathcal{X} : class(x) = c\}.$$

As with the univariate case, we can obtain these training features using a classification mask that masks out pixels belonging one class, for every class. The difference is that, in stead of gathering one feature

$[i, j]$ in every feature image, and collect them in a vector $x$ which is then included in $\mathcal{X}_c^t$. This is done for all pixels in the training mask for all the different classses.

We can now estimate the mean as

$$\hat{\mu}_c = \frac{1}{|\mathcal{X}_c^t|} \sum_{x \in \mathcal{X}_c^t} x$$

and the covariance matrix as

$$\hat{\Sigma}_c = \frac{1}{|\mathcal{X}_c^t|} \sum_{x \in \mathcal{X}_c^t} (x - \mu_c)(x - \mu_c)^\mathsf{T}$$

where $|\{\}|$ denotes the number of elements in the set. As before, we use hat notation to emphasize that these quantities are estimates for the true mean and covariance of the class (which is unknown).

The procedure for classifying an image $\mathcal{S}$ is then to classify each pixel $s \in \mathcal{S}$ by evaluating the function $a(x)$ for the feature vector assosiated with the pixel, and using the mean and covariance estimators obtained from the training.

For details on classification, dataset partition and evaluation, I refer to the notes on the univariate gaussian classifier.

## Singular covariance matrix

When classifying, one can encounter situations when the classifier breaks down, this section will cover one such case.

By definition, the covariance matrix is *non-negative definite*. To see this, consider an arbitrary set of $M$ vectors $x_i \in \mathbb{R}^N$ with a sample mean $\mu$, and arbitrary vector $v \in \mathbb{R}^N$, then

$$\begin{aligned} v^\mathsf{T} \Sigma v &= \frac{1}{M} \sum_{i=1}^{M} v^\mathsf{T} (x_i - \mu)(x_i - \mu)^\mathsf{T} v \\ &= \frac{1}{M} \sum_{i=1}^{M} [(x_i - \mu)^\mathsf{T} v][(x_i - \mu)^\mathsf{T} v] \\ &= \frac{1}{M} \sum_{i=1}^{M} [(x_i - \mu)^\mathsf{T} v]^2 \\ &\geq 0, \end{aligned}$$

which is true *iff* $\Sigma$ is non-negative definite.

However, it can still be *singular* (e.i. not invertible), a property it has if and only if its determinant is zero. This occurs if, for a class $c$ two or more of the feature images from that class are linearly dependent. To see this, consider a column $j$ of the covariance matrix from class $c$

$$\Sigma_{cj} = \frac{1}{|\mathcal{X}_c^t|} \sum_{x \in \mathcal{X}_c^t} (x - \mu_c)(x_j - \mu_{cj})$$

where $x_j$ is the $j^{th}$ entry (corresponding to feature $j$) of the feature vector $x$, and likewise with $\mu$. Now, if a set of columns $\{j_1, \ldots, j_k\}$ are to be linearly dependent, it implies that the the entries $\{x_{j_1}, \ldots, x_{j_k}\}$ are also linearly dependent. Linear dependent columns is one of many equivalent properties of a singular matrix, and the original statement is shown.

# Eigenvalues and eigenvectors

Without diving too deep into the fascinating world of eigenvectors and eigenvalues, we will present the basic concept, and how to compute it. In this course, we will mostly apply this theory to visualize feature clusters, via the eigenvectors and eigenvalues of the estimated covariance matrix of the class feature vectors. The covariance matrix contain information about the spread of the data, and since it is symmetric, we can gain information about its appearance (and hence, the appearance of the data) by studying the eigenvalues, and eigenvectors of the covariance matrix. I will not go into detail of why this is so, but rather refer the readers to this excellent explanation, and some beautiful visualizations. In the next paragraphs, I will mostly discuss how to actually compute the eigenvalues and eigenvvectors.

An eigenvector is a characteristic vector of a linear transformation that does not change its direction when the linear transformation is applied to it. A square matrix $A$ can represent a linear transformation, and in this case, we get the expression for its corresponding eigenvector $v$ and eigenvalue $\lambda$

$$Av = \lambda v.$$

This is equivalent to finding the roots of the equation

$$(A - \lambda I)v = 0, \qquad (1)$$

which has a non-zero solution $v$ if and only if the *determinant* of the matrix $A - \lambda I$ is zero (where $I$ denotes the identity)

$$\det(A - \lambda I) = 0.$$

Interested readers is referred to *wikipedia*, where the above paragraph is paraphrased from. This section is mostly included for reference, as we will be computing eigenvalues and eigenvectors in the assignments, and there it is assumed known.

## Computation in 2D

In the 2D case, with

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22}, \end{pmatrix}$$

we need to solve

$$A = \det \begin{pmatrix} a_{11} - \lambda & a_{12} \\ a_{21} & a_{22} - \lambda, \end{pmatrix} = 0$$

$$(a_{11} - \lambda)(a_{22} - \lambda) - a_{12}a_{21} = 0,$$

which is a regular second order equation w.r.t. $\lambda$, with solutions

$$\lambda^2 + (-a_{11} - a_{22})\lambda + (a_{11}a_{22} - a_{12}a_{21})$$

$$\begin{aligned}\lambda &= \frac{a_{11} + a_{22} \pm \sqrt{(a_{11} + a_{22})^2 - 4(a_{11}a_{22} - a_{12}a_{21})}}{2} \\ &= \frac{a_{11} + a_{22} \pm \sqrt{(a_{11} - a_{22})^2 + 4a_{12}a_{21})}}{2}.\end{aligned}$$

Evidently, we get two solutions, which we call $\lambda_1$ and $\lambda_2$, and these are the eigenvalues of $A$. We can substitute these back in equation (1), and find the corresponding eigenvectors $v_1$ and $v_2$.

Note that in the 2D case, we cannot be sure to have real eigenvalues.

•